



MINISTERUL EDUCAȚIEI, CULTURII ȘI CERCETĂRII

AL REPUBLICII MOLDOVA

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Informatică și Ingineria Sistemelor

Raport

pentru lucrarea de laborator Nr.1

la cursul de “Sisteme de Operare”

**Tema: Dezvoltarea unui mecanism de planificare a activităților de proces
pe GIT.**

A efectuat :

Siumbel Ela,
Zadorojneac Igor,
St. gr. SD-231,

A verificat:

Seniusin A.

Chișinău 2025

Содержание

Глава 1. Git: Основные команды и работа с ветками.....	5
Глава 2. Этапы работы.....	7
Часть 1. Организация рабочего процесса и контроль версий (Git).....	7
Часть 2. Анализ и оценка кода по критериям.....	8
Заключение.....	12

Введение

Настоящий отчет посвящен процессу разработки многофункционального Java-приложения, направленного на освоение ключевых концепций современного программирования, связанных с асинхронностью, планированием задач, многопоточностью и созданием графических интерфейсов. Основная цель работы заключалась в приобретении практических знаний и навыков, необходимых для проектирования отзывчивых и удобных приложений, а также в использовании системы контроля версий Git как инструмента профессиональной организации рабочего процесса.

В рамках работы были поставлены и реализованы следующие задачи:

- Освоение методов создания механизмов планирования (Timer, Thread): изучение и практическое применение инструментов Java для организации выполнения задач в заданные моменты времени или с определённой периодичностью.
- Освоение методов обработки механизмов многопоточности (минимум 3 потока): создание и управление несколькими потоками (Thread) для обеспечения параллельного выполнения задач.
- Создание и реализация программы с графическим интерфейсом (GUI): разработка интерактивного интерфейса средствами библиотеки Swing, позволяющего пользователю взаимодействовать с приложением.
- Использование платформы Git для контроля версий и организации рабочего процесса: фиксация изменений, работа с ветками (SD_231_6) и документирование истории проекта

Глава 1. Git: Основные команды и работа с ветками

В этой главе рассматриваются ключевые команды системы контроля версий Git, включая клонирование репозитория, добавление и коммит изменений, а также работа с ветками. Особое внимание уделено процессу создания, переименования и удаления веток, а также использованию `stash` и синхронизации с удалённым репозиторием.

1. Клонирование репозитория

`git clone <repo.name>` - Клонирование удалённого репозитория на компьютер.

2. Проверка состояния репозитория.

`git status` - Показывает изменённые, добавленные и неотслеживаемые файлы.

```
var
● amika@Amikas-MacBook-Air timer-app % git status
On branch SD_231_6
Your branch is up to date with 'origin/SD_231_6'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
       modified:   GUIApp.java
       modified:   MyProgram.java
       modified:   TimerApp.java
       modified:   TimerTasks.java

no changes added to commit (use "git add" and/or "git commit -a")
○ amika@Amikas-MacBook-Air timer-app %
```

Рисунок 1. Пример команд.

3. Добавление изменений

- Все файлы:

`git add .` - добавление всех файлов

```
● amika@Amikas-MacBook-Air java_project % git add .
● amika@Amikas-MacBook-Air java_project % git commit -m "interface and functionality update"
```

Рисунок 2. Пример команды `git add .`

- Конкретный файл: `git add <fileName>`

4. Создание коммита

- С сообщением: `git commit -m "Сообщение коммита"`

```
● amika@Amikas-MacBook-Air java_project % git add .
● amika@Amikas-MacBook-Air java_project % git commit -m "interface and functionality update"
[SD_231_6 5733828] interface and functionality update
2 files changed, 78 insertions(+), 53 deletions(-)
create mode 100644 .vscode/settings.json
○ amika@Amikas-MacBook-Air java_project %
```

Рисунок 3. Пример команды `git commit -m`

- Без сообщения (откроется редактор для ввода вручную): `git commit`
- Отправка изменений на удаленный репозиторий `git push`

5. Работа с ветками (Branches)

- Просмотр локальных веток `git branch`
- Переименование ветки `git branch -m <new_name>` - Переименовывает

локальную ветку.

- Удаление ветки: Локальная ветка: `git branch -d <branch_name>`
- Удалённая ветка: `git push origin --delete <branch_name>`
- Отправка новой ветки на удалённый репозиторий `git push origin -u`

`<branch_name>`

- **Stash:** временное сохранение изменений. Сохранить изменения `git stash -`

Замораживает текущие изменения, чтобы переключиться на другую ветку.

- Восстановить изменения `git stash pop` - Возвращает сохраненные

изменения обратно в рабочую ветку.

- Получение последних изменений с удалённого репозитория `git pull -`

Обновляет локальную ветку последними изменениями с удаленного репозитория.

```

● amika@Amikas-MacBook-Air java_project % git pull
remote: Enumerating objects: 23, done.
remote: Counting objects: 100% (16/16), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 23 (delta 4), reused 16 (delta 4), pack-reused 7 (from 1)
Unpacking objects: 100% (23/23), 7.75 KiB | 330.00 KiB/s, done.
From https://github.com/IrinaNicolaev/Testare
   da9a578..a135c9c  SD_231_5  -> origin/SD_231_5
   aeffd9..cd14b83  SD_231_7  -> origin/SD_231_7
Already up to date.
○ amika@Amikas-MacBook-Air java_project %

```

Рисунок 4. Пример команды `git pull`

Пример работы с ветками

1. Переименовать локальную ветку:

`git branch -m <new_name>`

2. Удалить старую ветку с удалённого репозитория:

`git push origin --delete <old_name>`

3. Отправить новую ветку на сервер:

git push origin -u <new_name>

Глава 2. Этапы работы

Часть 1. Организация рабочего процесса и контроль версий (Git)

Описывается структура рабочего процесса над проектом с использованием Git. Рассматривается порядок работы с ветками, последовательность коммитов, ведение истории изменений и организация совместной работы в команде.

Часть 2. Анализ и оценка кода по критериям

Проводится оценка качества кода по установленным критериям, включая читаемость, корректность, структуру и соблюдение стандартов. Рассматриваются методы проверки изменений, выявления ошибок и улучшения кода на основе обратной связи.

Часть 1. Организация рабочего процесса и контроль версий (Git)

Проект был инициализирован локально (**git init**) и привязан к удаленному репозиторию. Для организации разработки была создана изолированная ветка **SD_231_6**.

Процесс работы был зафиксирован в истории коммитов брэнча **SD_231_6**.

По 1 критерию : Зарегистрироваться на GitHub и создать локальный GIT-репозиторий.

Показатель	Значение
Репозиторий GitHub	https://github.com/IrinaNicolaev/Testare.git
Ветка разработки	SD_231_6

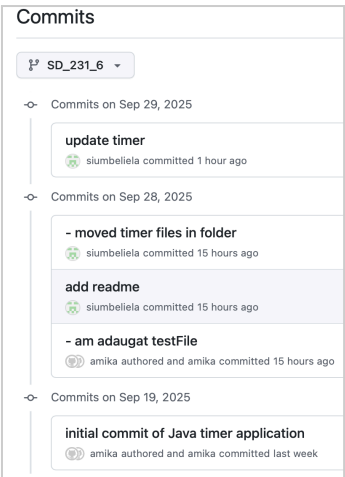


Рисунок 5. Скриншот коммитов.

Этап	Дата	Бренч	Коммит / Команда	Комментарий
Инициализация	19.09.2025	(Основной)	initial commit of Java timer application	Базовый код проекта (автор: amika).
Создание бранча	28.09.2025	SD_231_6	Создание бранча	Создан отдельный бранч для изолированной разработки по заданию.
Структурирование	28.09.2025	SD_231_6	moved timer files in folder, add readme	Реорганизация репозитория, добавление документации.
Разработка GUI	29.09.2025	SD_231_6	update timer	Разработка логики таймеров и потоков.
Финальный коммит	29.09.2025	SD_231_6	interface and functionality update	Фиксация финального кода GUI (GUIApp.java) и завершение функциональности. Подтверждено командами <code>git add</code> и <code>git commit</code> .

Разработка велась в отдельном рабочем бранче, и все этапы зафиксированы в истории коммитов.

Часть 2. Анализ и оценка кода по критериям

Анализ основан на представленных файлах: `MyProgram.java`, `TimerApp.java`, `GUIApp.java`, `TimerTasks.java`.

Критерий 2: Создать и инициализировать таймер.

(файл `MyProgram.java`): В коде инициализированы и используются три три различных сценария работы таймера:

- **Периодический таймер (`soundTimer`)** – воспроизводит звук каждые 2 секунды. Это демонстрирует работу таймера с фиксированным интервалом.
- **Одноразовый таймер (`messageTimer`)** – через 5 секунд после запуска программы отменяет выполнение первого таймера. Таким образом, показана возможность завершать задачи по времени.
- **Таймер по расписанию (`clock`)** – запускает задачу точно в указанное время. Это позволяет планировать выполнение задач в будущем.

Использование задач (`TimerTask`)

- Созданы два класса-наследника `TimerTask`:
 - `SoundPlayer` – воспроизводит системный звук (`Toolkit.getDefaultToolkit().beep()`).
 - `Message` – выводит в консоль текстовое сообщение.
- Также используется анонимный класс `TimerTask` для отмены первого таймера.

Периодическое выполнение

- С помощью `soundTimer.scheduleAtFixedRate(...)` реализован вызов метода `run()` каждые 2 секунды.

Однократное выполнение

- `messageTimer.schedule(...)` запускает задачу через 5 секунд после старта программы и завершает работу `soundTimer`.

Выполнение по расписанию (на определённую дату/время)

- С помощью `Calendar` формируется объект `Date` с точным временем (18:24:00).
- `clock.schedule(...)` запускает задачу именно в этот момент.

Результат программы


```
orage/428/e9d22f71cd54fdb47c44036a3c88/redhat.java
Start
Sound played
Sound played
Sound played
5 seconds have passed. Cancelling sound.
```

Рисунок 6. Результат вывода программы.

Критерий 3: Создайте не менее 3 потоков в приложении.

- (файл **TimerApp.java**): В приложении созданы и запущены три объекта

Thread:

1. **soundThread** (для задачи **SoundPlayer**).

Реализует интерфейс *Runnable* и выводит сообщение о воспроизведении звука.

2. **messageThread** (для задачи **Message**).

Реализует интерфейс *Runnable* и выводит заданное при создании объекта сообщение.

3. **thirdThread** (анонимный поток).

Данный поток реализован как анонимный класс (через лямбда), демонстрирующий параллельный запуск кода.

Запуск потоков

- Все три объекта **Thread** были инициализированы и запущены методом **start()**.
- После запуска потоки выполняются параллельно, что подтверждает многопоточность программы.

Результаты выполнения программы

После запуска **TimerApp.java** в консоли получен следующий вывод:

```
Main Thread started. Starting 3 new threads.
All threads started. Main thread finished.
This is Thread 2's scheduled message!
Thread 3 (Anonymous): I am the third thread, running in parallel.
Thread 1 (SoundPlayer): Playing sound (simulated).
amika@Amikas-MacBook-Air java_project %
```

Рисунок 7. Результат вывода программы.

Критерий 4: Создать интерфейс программы.

- (файл **GUIApp.java**): Приложение использует библиотеку **Swing** для создания графического интерфейса пользователя (GUI). Интерфейс включает:

- Основное окно (**JFrame**).
- Компоненты ввода (**TextField**) для трех числовых значений.
- Интерактивные кнопки (**Button**) "Calculate Sum" и "Reset".
- Обработка событий и вывод результата.

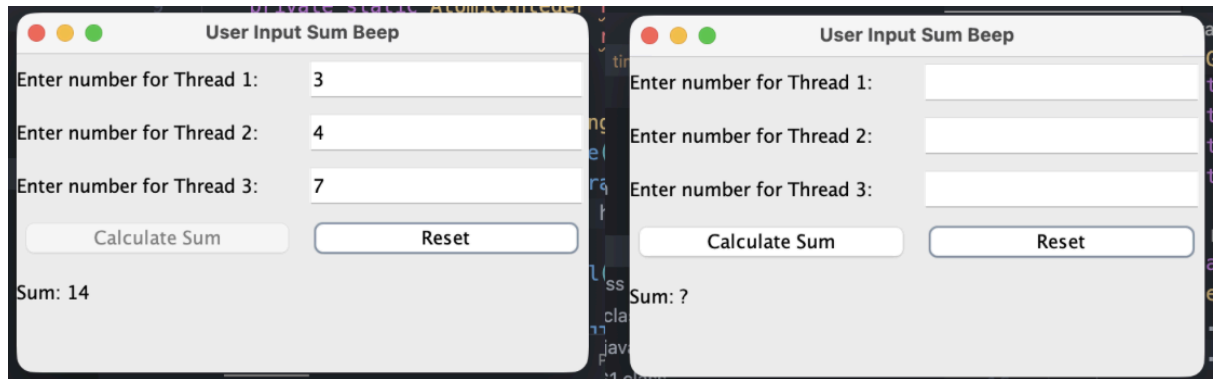


Рисунок 8. Интерфейс программы.

Критерий 5: Корректность кода и отсутствие ошибок.

- **MyProgram.java** – созданы и инициализированы три таймера (**soundTimer**, **messageTimer**, **clock**) для разных задач: периодическое выполнение, одноразовый запуск и запуск по времени.
- **TimerApp.java** – реализованы три потока (**soundThread**, **messageThread**, **thirdThread**), которые выполняются параллельно, что подтверждает работу многопоточности.
- **TimerTasks.java** – используется **SwingUtilities.invokeLater()** для безопасного обновления GUI из потоков, предусмотрена обработка исключений, что обеспечивает корректность и стабильность работы программы.
- **GUIApp.java** – создан графический интерфейс с вводом чисел, вычислением суммы и кнопкой сброса. Реализована обработка ошибок ввода и системный beep для обратной связи пользователю.
- **Итог:** все критерии (таймеры, потоки, корректность кода, GUI) выполнены, программа работает стабильно и соответствует требованиям.

Критерий 6: Соответствие инструкциям и требованиям.

- **Сводка:**
- **Таймеры:** Создано 3 таймера (**MyProgram.java**).

- **Потоки:** Создано 3 явных потока (`TimerApp.java`).
- **Интерфейс:** GUI реализован (`GUIApp.java`).

Критерий 7: Оптимизация кода.

В проектах реализована базовая оптимизация:

- Используются отдельные классы (`SoundPlayer`, `Message`) для повторно используемых задач, что уменьшает дублирование кода.
- Применён `SwingUtilities.invokeLater()` для безопасного обновления GUI, исключая потенциальные ошибки и повышая стабильность.
- В `GUIApp.java` введены `AtomicInteger` для потокобезопасного хранения данных.
- Предусмотрена обработка исключений при вводе, что предотвращает аварийное завершение программы.
- Логика вынесена в отдельные методы (`parseInput`), что повышает читаемость и упрощает поддержку кода.

Оптимизация достигнута за счет структурирования, повторного использования классов и обработки исключений. Код остаётся устойчивым, понятным и легко расширяемым.

Заключение

Выполнение работы позволило успешно достичь поставленных целей и получить комплексное представление о процессах, лежащих в основе разработки современных приложений на Java. В ходе практической части были освоены методы планирования задач с использованием классов **Timer** и **TimerTask**, что дало возможность управлять временем выполнения операций и обеспечивать гибкость работы приложения.

Значительное внимание было уделено **многопоточности**. Создание и управление не менее чем тремя потоками позволило на практике отработать принципы асинхронного программирования, а также понять важность синхронизации и корректного распределения ресурсов между задачами.

Важным этапом стало создание **графического интерфейса (GUI)** средствами библиотеки **Swing**, что позволило объединить техническую часть проекта с удобным пользовательским взаимодействием. Работа с интерфейсом научила учитывать не только функциональность, но и удобство конечного пользователя.

Неотъемлемой частью процесса стало применение системы контроля версий **Git**, обеспечившей грамотную организацию разработки. Работа с репозиторием, ветвлением и фиксацией изменений продемонстрировала преимущества командной и индивидуальной разработки в условиях реальных проектов.

Таким образом, работа носила комплексный характер, сочетая теорию и практику. За короткий срок удалось не только углубить знания в области Java, но и закрепить навыки работы с инструментами, необходимыми для профессиональной разработки: многопоточностью, планировщиками задач, пользовательскими интерфейсами и Git. Полученный опыт является прочной основой для дальнейшего изучения и реализации более масштабных проектов в сфере программной инженерии.

Приложения

// Файл: MyProgram.java

```
import java.awt.Toolkit;
import java.util.Calendar;
import java.util.Date;
import java.util.Timer;
import java.util.TimerTask;

/**
 * Класс, который воспроизводит звук.
 */
class SoundPlayer extends TimerTask {
    @Override
    public void run() {
        Toolkit.getDefaultToolkit().beep();
        System.out.println("Sound played");
    }
}

/**
 * Класс, который выводит сообщение.
 */
class Message extends TimerTask {
    String msg;

    public Message(String msg) {
        this.msg = msg;
    }

    @Override
    public void run() {
        System.out.println(msg);
    }
}

public class MyProgram {

    public static void main(String[] args) {
        System.out.println("Start");

        // Таймер для звука (каждые 2 секунды)
        Timer soundTimer = new Timer();
        TimerTask soundPlayer = new SoundPlayer();
        soundTimer.scheduleAtFixedRate(soundPlayer, 0, 2000);

        // Таймер, который отменяет первый таймер через 5 секунд
        Timer messageTimer = new Timer();
        messageTimer.schedule(new TimerTask() {
            @Override
            public void run() {
                System.out.println("5 seconds have passed. Cancelling sound.");
                soundTimer.cancel();
            }
        }, 5000);
    }
}
```

```

    }, 5000);

    // Таймер для сообщения в определенное время
    Calendar calendar = Calendar.getInstance();
    calendar.set(Calendar.HOUR_OF_DAY, 18);
    calendar.set(Calendar.MINUTE, 24);
    calendar.set(Calendar.SECOND, 0);
    Date date = calendar.getTime();

    Timer clock = new Timer();
    TimerTask message = new Message("Salut, Salut, Salut!!!");
    clock.schedule(message, date);
}
}

// Файл: TimerTasks.java
import java.awt.Toolkit;
import javax.swing.JLabel;
import javax.swing.SwingUtilities;

// Класс для звука и обновления статуса
class SoundPlayer implements Runnable {
    private JLabel statusLabel;

    public SoundPlayer(JLabel label) {
        this.statusLabel = label;
    }

    @Override
    public void run() {
        for (int i = 1; i <= 5; i++) {
            final int count = i;
            Toolkit.getDefaultToolkit().beep();
            System.out.println("Thread 1: Sound played, count: " + count);

            // !!! БЕЗОПАСНОЕ ОБНОВЛЕНИЕ GUI !!!
            SwingUtilities.invokeLater(() -> {
                statusLabel.setText("Thread 1 (Sound): Playing... (" + count +
"/5)");
            });

            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                // ...
            }
        }
        SwingUtilities.invokeLater(() -> statusLabel.setText("Thread 1 (Sound):
FINISHED."));
    }
}

// Класс для сообщения и обновления статуса
class Message implements Runnable {
    private String msg;

```

```

private JLabel statusLabel;

public Message(String msg, JLabel label) {
    this.msg = msg;
    this.statusLabel = label;
}

@Override
public void run() {
    System.out.println("Thread 2: " + msg);
    SwingUtilities.invokeLater(() -> statusLabel.setText("Thread 2
(Message): " + msg));
}
}

// Файл: TimerApp.java

public class TimerApp {
    public static void main(String[] args) {
        System.out.println("Main Thread started. Starting 3 new threads.");

        // 1. Создаем первый поток для звука
        Thread soundThread = new Thread(new SoundPlayer());

        // 2. Создаем второй поток для сообщения
        Thread messageThread = new Thread(new Message("This is Thread 2's
scheduled message!"));
        // 3. Создаем третий, анонимный поток для демонстрации третьего потока

        Thread thirdThread = new Thread(() ->
            System.out.println("Thread 3 (Anonymous): I am the third thread,
running in parallel.")
        );

        // Запускаем все три потока
        soundThread.start();
        messageThread.start();
        thirdThread.start();

        System.out.println("All threads started. Main thread finished.");
        // Задачи будут выполняться параллельно
    }

    // SoundPlayer class implementing Runnable
    static class SoundPlayer implements Runnable {
        public SoundPlayer() {
            // No-argument constructor
        }

        @Override
        public void run() {
            System.out.println("Thread 1 (SoundPlayer): Playing sound
(simulated).");
        }
    }
}

```

```

}

// Message class implementing Runnable
class Message implements Runnable {
    private String message;

    public Message(String message) {
        this.message = message;
    }

    @Override
    public void run() {
        System.out.println(message);
    }
}

```

// Файл: TimerApp.java GUIApp

```

import java.awt.*;
import java.util.concurrent.atomic.AtomicInteger;
import javax.swing.*;

public class GUIApp{

    private static JLabel sumLabel = new JLabel("Sum: ?");
    private static AtomicInteger result1 = new AtomicInteger(0);
    private static AtomicInteger result2 = new AtomicInteger(0);
    private static AtomicInteger result3 = new AtomicInteger(0);

    public static void main(String[] args) {
        JFrame frame = new JFrame("User Input Sum Beep");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(400, 250);

        JPanel panel = new JPanel(new GridLayout(6, 2, 5, 5));

        JTextField input1 = new JTextField();
        JTextField input2 = new JTextField();
        JTextField input3 = new JTextField();

        JButton calculateButton = new JButton("Calculate Sum");
        JButton resetButton = new JButton("Reset");

        panel.add(new JLabel("Enter number for Thread 1:"));
        panel.add(input1);
        panel.add(new JLabel("Enter number for Thread 2:"));
        panel.add(input2);
        panel.add(new JLabel("Enter number for Thread 3:"));
        panel.add(input3);

        panel.add(calculateButton);
        panel.add(resetButton);

        panel.add(sumLabel);
        panel.add(new JLabel()); // пустая ячейка
    }
}

```



```

// Кнопка расчёта суммы
calculateButton.addActionListener(e -> {
    calculateButton.setEnabled(false);
    int num1 = parseInput(input1.getText());
    int num2 = parseInput(input2.getText());
    int num3 = parseInput(input3.getText());

    result1.set(num1);
    result2.set(num2);
    result3.set(num3);

    int sum = result1.get() + result2.get() + result3.get();
    sumLabel.setText("Sum: " + sum);

// Системный beep на любой результат
Toolkit.getDefaultToolkit().beep();
System.out.println("Beep! Sum = " + sum);
});

// Кнопка сброса
resetButton.addActionListener(e -> {
    input1.setText("");
    input2.setText("");
    input3.setText("");
    sumLabel.setText("Sum: ?");
    result1.set(0);
    result2.set(0);
    result3.set(0);
    calculateButton.setEnabled(true);
    // Системный beep при сбросе
    Toolkit.getDefaultToolkit().beep();
    System.out.println("Beep! Reset pressed");

});

frame.add(panel);
frame.setVisible(true);
}

private static int parseInput(String text) {
    try {
        return Integer.parseInt(text.trim());
    } catch (NumberFormatException e) {
        return 0;
    }
}
}

```